

# Answer Key & Marking Guide

Teacher copy - do not share with the student

Marking philosophy: give credit for **correct logic** even with small syntax slips. A student who writes `df['Rating'].max` (missing brackets) understood the idea - give most marks. Different valid approaches deserve full marks.

## Section A - MCQ (1 mark each)

- A1. (b) (40, 9) - rows first, then columns
- A2. (c) `df['Rating'].mean()`
- A3. (b) All free games
- A4. (a) `value_counts()`
- A5. (c) Scatter plot
- A6. (b) Removes rows with missing values

## Section B - Pandas

### B1. Explore the data [3] - 1 mark each part

```
df.shape                # (40, 9)
df.isna().sum()         # Rating: 2, Hours_To_Beat: 1
df.loc[df['Rating'].idxmax(), 'Title'] # 'Block Builder' (9.5)
# also fine: df.sort_values('Rating', ascending=False).head(1)
```

### B2. Filter [4] - 1 mark per condition combined correctly (3), 1 for column selection

```
mask = (df['Multiplayer'] == 'Yes') & (df['Price'] < 40) & (df['Rating'] >= 8)
result = df[mask][['Title', 'Price', 'Rating']]
print(len(result))    # 10 games
result
# Expected: Pixel Racer, Block Builder, Galaxy Wars, Castle Siege, Dungeon Dive,
#           Cooking Craze, Moon Miner, Rhythm Rush, Fairy Village, Chess Champs
```

Common mistake: using `and` instead of `&`, or forgetting brackets around each condition. If they got the idea but hit this error, give 3/4.

### B3. Group and compare [5] - (a) 2, (b) 1, (c) 2

```
df.groupby('Genre')['Copies_Sold_Millions'].mean().sort_values(ascending=False)
# Puzzle (~23.5) and Sandbox (~23.25) are top
```

```
df['Platform'].value_counts()
# Switch 9, PC 9, Mobile 9, Xbox 7, PlayStation 6 -> a three-way tie!
# .idxmax() returns 'Switch' (first one). Accept any of the three;
# bonus praise if the student notices the tie.
```

```
df.groupby('Platform')['Price'].max()
```

### B4. New columns [5] - (a) 1, (b) 1, (c) 1, (d) 2

```
df['Revenue_Millions'] = df['Price'] * df['Copies_Sold_Millions']
df['Age'] = 2026 - df['Release_Year']
df['Is_Free'] = df['Price'] == 0
df.sort_values('Revenue_Millions', ascending=False).head(5)
# Top: Kart Kings (1289.8), Block Builder (1131.9), Goal Masters (489.9),
#     Mystic Forest (431.9), Galaxy Wars (339.9)
```

### B5. Clean the data [5] - (a) 1, (b) 1, (c) 1, (d) 2 for explanation

```
df['Rating'] = df['Rating'].fillna(df['Rating'].mean())
df['Hours_To_Beat'] = df['Hours_To_Beat'].fillna(df['Hours_To_Beat'].median())
df = df[df['Hours_To_Beat'] != 999]
```

```
# (d) The mean is pulled way up by the 999 outlier (mean ~48 vs median 14),
#     so the median is a fairer "typical" value.
```

Full marks for (d) if the student mentions the 999 outlier dragging the mean. This is the key insight of the whole section.

## Section C - Matplotlib

Rubric for every plot: correct plot type (1-2), correct data (1), title + labels (1), extra requirement (1).

### C1. Bar chart [4]

```
counts = df['Genre'].value_counts()
plt.bar(counts.index, counts.values)
plt.title('Games per Genre')
plt.xlabel('Genre'); plt.ylabel('Number of games')
plt.xticks(rotation=45)
plt.show()
# Also fine: counts.plot(kind='bar')
```

### C2. Scatter [5] - plot 3, interpretation 2

```
plt.scatter(df['Price'], df['Copies_Sold_Millions'])
plt.title('Price vs Copies Sold'); plt.xlabel('Price ($)'); plt.ylabel('Copies sold (millions)')
plt.show()
# Interpretation: cheaper/free games generally sell more; free games (Price=0)
# cluster on the left with very high sales (Candy Crusher 55M). Kart Kings and
# Block Builder are exceptions.
```

### C3. Subplots [5] - subplots setup 1, histogram 2, line plot 2

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(df['Rating'], bins=10)
axes[0].set_title('Rating distribution'); axes[0].set_xlabel('Rating'); axes[0].set_ylabel('Count')
per_year = df['Release_Year'].value_counts().sort_index()
axes[1].plot(per_year.index, per_year.values, marker='o')
axes[1].set_title('Games released per year'); axes[1].set_xlabel('Year'); axes[1].set_ylabel('Games')
plt.tight_layout(); plt.show()
```

## Section D - Seaborn

### D1. Countplot [4] - plot 2, hue 1, comment 1

```
sns.countplot(data=df, x='Platform', hue='Multiplayer')
plt.title('Games per Platform'); plt.show()
# Mobile has the most single-player (No) games.
```

### D2. Boxplot [5] - plot 3, two observations 2

```
sns.boxplot(data=df, x='Genre', y='Rating')
plt.xticks(rotation=45); plt.title('Rating by Genre'); plt.show()
# Highest median: RPG (8.95), then Sandbox (8.85). Widest spread: Simulation (6.7 to 8.9).
# Accept any answer that matches what their plot shows.
```

### D3. Heatmap [5] - corr 1, heatmap 2, interpretation 2

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation heatmap'); plt.show()
# On the ORIGINAL data: Release_Year vs Copies_Sold_Millions, negative (-0.49)
# (older games have had more time to sell). Close second: Hours_To_Beat vs
# Copies_Sold (+0.45), inflated by the 999 outlier.
# If they removed the 999 row in B5: Rating vs Hours_To_Beat, positive (+0.60).
# Accept whichever matches THEIR heatmap - checking against their own plot is the skill.
```

## Bonus [4]

Any evidence-based answer. A strong answer: *Free or cheap Puzzle/Sandbox game on Mobile or PC, because free games have the highest average copies sold and Mobile has the biggest sellers.*

Also strong: *Racing on Switch (Kart Kings)* shows *premium games can sell huge too*. Award marks for: uses numbers/plots (2), gives a clear recommendation (1), covers all four factors (1).

## Grading scale (suggested)

| Marks | Level         | What it tells you                                    |
|-------|---------------|------------------------------------------------------|
| 50-60 | Excellent     | Ready for scikit-learn / real ML                     |
| 38-49 | Good          | Solid; revisit groupby and cleaning                  |
| 25-37 | Developing    | Redo filtering & plotting labs before moving on      |
| < 25  | Needs support | Go back to pandas basics with more hands-on practice |